



Week 3 — JavaScript Core Concepts

A Practice & Revision Week

Week 3

Issued: 12 July 2026

Distributed as `Week-03-JavaScript-Core-Concepts.pdf` in the TTT-Interns repo, together with the `drills/` practice kit.

Timeline

- **Issued:** Sunday, 12 July 2026
- **Work begins:** Monday, 13 July 2026
- **Practice due:** Thursday, 16 July 2026 — push your work before the end of the day.
- **Friday, 17 July: Test**, followed by discussion and doubt-clearing.

There is a test on Friday

It covers the topics listed in this document, and nothing outside them. Study accordingly.

Overview

This is a **practice and revision week**. There is no application to build — no project, no UI, no README to write at midnight.

The week has a single purpose: to make you genuinely comfortable with the JavaScript that every remaining week of this program is built on. Week 2 covered a great deal of ground quickly, and this week gives that material the time it deserves, alongside a few new concepts that go a level deeper.

The topics are broken into four daily blocks. The practice comes as a set of small functions to write, with a test suite that checks your work as you go, so you always know exactly where you stand.

Prerequisite: Week 2. If any part of it felt rushed, this is the week to close that gap.

Learning Objectives

By the end of this week you should be comfortable:

- Using `let`, `const`, arrow functions, template literals, destructuring, spread and rest without having to look them up.
- Transforming data with `map`, `filter`, `reduce`, `find`, `some` and `every`.
- Writing **pure functions** — small functions that take input and return output.

- Explaining what a **closure** is, and writing functions that return other functions.
- Updating arrays and objects **without changing the original** (immutability).
- Reading nested data safely with `?.` and `??`.
- Working with promises, `async` / `await`, `try` / `catch`, and `Promise.all`.
- Explaining each of these ideas in your own words.

The Topics

Block 1 — The Basics, Properly (Monday)

Topic	What to be comfortable with
<code>let</code> and <code>const</code>	when to use which · block scope · why <code>var</code> is avoided
Functions	declarations vs expressions · parameters · return values
Arrow functions	short syntax · implicit return · when to use them
Default parameters	<code>function greet(name = 'friend')</code>
Template literals	<code>`Hello, \${name}`</code> · multi-line strings
Destructuring	pulling values out of objects and arrays · renaming · defaults
Spread & rest	copying and merging (<code>...</code>) · collecting arguments (<code>...args</code>)
Optional chaining <code>?.</code>	reading <code>user.address.city</code> when <code>address</code> might not exist
Nullish coalescing <code>??</code>	supplying a default — and how it differs from <code> </code>

Block 2 — Working with Data (Tuesday)

Topic	What to be comfortable with
<code>map</code>	transform every item into something new
<code>filter</code>	keep only the items that match
<code>find</code>	get the first match — and what it returns when there is none
<code>reduce</code>	reduce a list to a single value: a total, an average, a count
<code>some</code> / <code>every</code>	"is any...?" and "are all...?"
Chaining	<code>filter(...).map(...)</code> reading left to right
Immutable updates	adding, updating and removing from a list by building a new array, not editing the old one
The empty case	what your function should do when the list is empty

`reduce` is the one most people find hardest. Give it real time. Once it clicks, it stays clicked.

Block 3 — Functions, a Level Deeper (Wednesday)

Topic	What to be comfortable with
Functions as values	passing a function as an argument · returning a function
Higher-order functions	a function that takes or returns another function
Closures	a function that "remembers" the variables around it
Pure functions	same input → same output · no side effects

Block 4 — Asynchronous JavaScript (Thursday)

Topic	What to be comfortable with
Why async exists	code that takes time · why the page must not freeze
Promises	pending · fulfilled · rejected · <code>.then</code> / <code>.catch</code>
<code>async</code> / <code>await</code>	the readable way to write the same thing
<code>try</code> / <code>catch</code> / <code>finally</code>	handling failure without crashing
<code>Promise.all</code>	doing several things at the same time
Sequential vs parallel	why <code>await</code> inside a loop is slower, and when that is acceptable

Your Work This Week

Part 1 — The Practice Drills

In the `drills/` folder you will find five files of small functions to write, and a test suite that checks them.

```
cd drills
npm test          # runs all the checks
```

Everything starts out failing. As you complete each function, the checks turn green. That immediate feedback is the point — you never have to wonder whether you got it right.

File	What you will practise
<code>01-basics.js</code>	template literals · destructuring · spread · rest · default parameters · <code>?.</code> · <code>??</code>
<code>02-arrays.js</code>	<code>map</code> · <code>filter</code> · <code>find</code> · <code>reduce</code> · <code>some</code> / <code>every</code>
<code>03-objects.js</code>	immutable updates · optional chaining · <code>??</code>
<code>04-functions.js</code>	higher-order functions · returning functions · closures
<code>05-async.js</code>	<code>async</code> / <code>await</code> · <code>try</code> / <code>catch</code> · <code>Promise.all</code> · parallel vs sequential

Five or six short functions per file. Work through them **in order** — each file builds on the one before.

Every function has a comment above it explaining what it should do, with examples. If a comment is unclear, ask — that is a problem with our explanation, not with you.

Part 2 — Your Notes (`NOTES.md`)

As you work, write short notes in `drills/NOTES.md` — a few sentences each, **in your own words**:

1. What is the difference between `let` and `const`? When would you use each?
2. Explain `map`, `filter` and `reduce` to someone who has never seen them. One sentence each.
3. What is a **closure**? Use an example from your own drill code.
4. What does an "immutable update" mean, and why not simply change the original array?
5. What is the difference between `value ?? fallback` and `value || fallback`? When does it matter?
6. What does `await` actually do? What is the difference between doing three things one after another and doing all three at once?

Write these yourself — the aim is for the understanding to end up in your head. If an answer feels incomplete, say so honestly; a gap you have identified is far more useful than one you have papered over.

Optional — `06-stretch.js` ☀️

A few harder challenges for anyone who finishes early. **Entirely optional, and they carry no marks.** Run them with `npm run test:stretch`.

Suggested Plan — 4 hours a day

Day	Study	Practice	Roughly
Mon 13	Block 1 — the basics, properly	<code>01-basics.js</code> (5 functions)	3 hrs
Tue 14	Block 2 — working with data	<code>02-arrays.js</code> (6) + <code>03-objects.js</code> (5)	4½ hrs
Wed 15	Block 3 — functions & closures	<code>04-functions.js</code> (5)	4 hrs
Thu 16	Block 4 — asynchronous JavaScript	<code>05-async.js</code> (4) · finish <code>NOTES.md</code> · push	4½ hrs
Fri 17	Test , then discussion and doubt-clearing	—	—

A few notes on the pacing, so you can plan properly:

- **Monday is deliberately the lightest day.** Block 1 is mostly revision of Week 2, and the five functions in `01-basics.js` are short. If you finish early, start reading Block 2 — Tuesday is the busiest day, and an hour of head start there is an hour well spent.
- **Tuesday carries two drill files, but the second one is the shortest.** `03-objects.js` reuses exactly what you learn on Monday and Tuesday — spread, `map`, `filter`, `?.` and `??` — so once those have landed, its five functions go quickly. Do `02-arrays.js` first.
- **`reduce` is the single hardest thing this week.** It is worth half of Tuesday on its own. If it takes longer, let it.
- **Wednesday has only one file, on purpose.** Closures are the one genuinely new idea of the week, and they deserve an undivided day. Don't rush it.

This is a suggestion, not a rule. Understanding five things properly is worth more than half-understanding fifteen.

If you are stuck for more than 30–45 minutes, ask. Show what you have tried and where the confusion is. Asking early is a strength, and it is always better than losing an evening to it.

Submission

- Create a public repository — `javascript-practice` is a suitable name.
- Include the completed `drills/` folder and your `NOTES.md`.
- **Commit as you go.** One commit per drill file is a good rhythm, with meaningful messages.
- A short `README.md`: what you practised, what you found difficult, and what finally made sense.
- Push before the end of **Thursday 16 July**, and share the repository link.

No screenshots, no polish, no long write-up required. This week, your work is your learning.

Assessment

	Marks
Practice drills — checks passing	40
<code>NOTES.md</code> — the concepts, in your own words	15
Friday test	40
Commits & repository tidiness	5
Total	100

This is a learning week and it is marked as one. There is nothing to build and no polish to score — the hours you put into understanding the material are what these numbers measure.

If a topic still feels shaky by Friday, say so. That is useful information, and Friday's discussion exists precisely to resolve it.